Phase 3: Testing Plan

I. Team Details

Application Name: Streamly
Primary Customer: Daniyal
Next Customer Meeting Date: 3 April 2026
Team Members:

- Devin Boodoo
- Johnathan Lavoie
- Maryam Hameed
- Ashwin Sivasakthi
- Avis Shrestha

II. Overview

Streamly is a music library catalogue system inspired by platforms such as Spotify and Youtube Music. The purpose of the system is to provide users with an organized, filterable and searchable music database while allowing administrators full control over managing the underlying music and song records.

That being said, this catalogue system requires that two main areas should follow through testing. The first is the java application layer, which includes the core domain classes such as User, Playlist, Song and Album. The second is the TypeScript authentications service, which has the responsibility of exposing routes for registration, login and retrieving the current authenticated user. The Java portion models the music catalogue and user data, while the TypeScript service handles secure authentication, a JSON-based persistence and token verification.

Streamly's overall testing approach is layered, by which unit tests will validate individual methods, JSON utility functions and route behaviour. Integration tests verify interactions between express routes, JSON file persistence, password hashing, and JWT-based authentication. The system tests validate the user workflows, especially the authentication process and the core music-library behaviour. Hence, this would give coverage from small components up to complete end-to-end scenarios.

For automation, our testing plan uses JUnit for the Java classes since the Java project is Maven-based and contains Java domain objects. The TypeScript service should use an automated HTTP or API testing setup such as Node/Bun-compatable test runner together with route-level testing.

All three testing views are applied where appropriate

- Clear Box (CB): branches, data handling logic, and internal structure
- Translucent Box (TB): partial knowledge of implementation, especially for file persistence.
- OB (Opaque Box): external user behaviour through public interfaces and workflows

III. Unit Tests

These unit tests focus on single methods, classes in isolation and route handlers.

Test ID	Method/Class	Input(s)	Expected Output	Testing Approach (Manual/Automated)	Assigned Team Member
UT-01-CB	`userExists()`	Existing username in users.json	`true`	Automated	Maryam
UT-02-CB	`userExists()`	Non-existing username	`false`	Automated	Maryam
UT-03-CB	`getUserByUserName()`	Valid username	Returns correct user object	Automated	Maryam
UT-04-CB	`getUserByUserName()`	Invalid username	`null`	Automated	Maryam
UT-05-CB	`writeUser()`	New user object	User is appended to JSON file	Automated	Maryam
UT-06-TB	`/auth/register`	New username & password	Success message (status 201)	Automated	N/A
UT-07-TB	`/auth/register`	Duplicated username + password	Message displayed “user already exists” (status 400)	Automated	N/A
UT-08-TB	`/auth/login`	Valid username + correct password	“Login Successful” (status 200)	Automated	N/A
UT-09-TB	`/auth/login`	Valid username + wrong password	“Invalid credentials” (status 401)	Automated	N/A
UT-10-TB	`/auth/login`	Unknown username + password	“Invalid credentials” (status 400)	Automated	N/A
UT-11-TB	`/auth/me`	Valid token	`Returns userId, username, isAdmin`	Automated	N/A
UT-12-TB	`/auth/me`	No token	“Not authenticated”	Automated	N/A

			(status 401)		
UT-13-TB	`/auth/me`	Invalid token	"Invalid token" (status 401)	Automated	Ashwin
UT-14-OB	`Playlist.getName()`	Initialized playlist object	Returns the stores field value correctly	Automated (JUnit)	Maryam
UT-15-OB	`User.isAdmin()`	User with admin false	Returns `false`	Automated (JUnit)	Maryam
UT-16-OB	`User.setUsername()` + `getUsername()`	Set "`user123`"	Getter would return "`user123`"	Automated (JUnit)	Maryam
UT-17-CB	`Playlist.setSongIds()` + `getSongIds()`	Array of song IDs	Same array is returned	Automated (JUnit)	Maryam
UT-18-CB	`Song.from_file()`	Valid song JSON file	Returns a populated `Song` object	Automated (JUnit)	Ashwin
UT-19-CB	`Song.from_file()`	Malformed JSON	`null`	Automated (JUnit)	Ashwin
UT-20-CB	`save_to_file()`	Valid `Song` object	JSON written with the current fields	Automated (JUnit)	Ashwin
UT-21-OB	`filter.scoreAny()`	Song with default parameters and an empty filter	A score of 0 (passed all criteria / song is a perfect match)	Automated (JUnit)	Johnathan
UT-22-OB	`filter.scoreAny()`	Song with null parameters and a default filter	A score of 0 (passed all criteria / song is a perfect match)	Automated (JUnit)	Johnathan
UT-23-OB	`filter.scoreAny()`	Two songs with default parameters and a default filter with and without weights	The filter with weights should be more strict and have a higher score	Automated (JUnit)	Johnathan
UT-24-OB	`filter.scoreAny()`	Two songs with default parameters and a default filter close to song_1's title	Song_1 should have a lower score (more accurate match) compared to song_2	Automated (JUnit)	Johnathan

UT-25-OB	`filter.scoreAny()`	Two songs with default parameters and a default filter set to song_1's title	Song_1 should have a score of 0 (perfect match) compared to song_2 which should be higher	Automated (JUnit)	Johnathan

Unit Testing Elaboration: The authentication tests are essential because the TypeScript code contains direct control flow for checking whether a user exists, hashing passwords, reading a stored user, and writing JSON-backed persistence. The Java unit tests are equally as important even though some of the model classes mostly contain getters and setters. Because of this they verify the integrity of the object, which is foundational for the functionality of music metadata, search, playlists and favourites.

Supported Unit Test and File Mapping Justification:

File Mapping	Tests come directly from:	Relevant Unit Tests	Justification
JSON Utility Functions (Core Backend Logic)	json.ts	UT-01-CB UT-02-CB UT-03-CB UT-04-CB UT-05-CB	Reads users from JSON. Checks existence. Writes users to storage. Foundation of authentication data logic.
Authentication Route: Login	login.ts	UT-08-TB UT-09-TB UT-10-TB	Password validation via comparing hashes. Error handling for invalid credentials.
Authentication Route: Registration	register.ts	UT-06-TB UT-07-TB	Responsible for creating new users. Hashing passwords via bcrypt. Writing user data to store.
Authentication Route: Pre-existing registered user	me.ts	UT-11-TB UT-12-TB UT-13-TB	Ensures that there is secure session handling. Proper token validation. Protection of restricted areas. Highly important for security.
Java Domain Classes: User Class	User.java	UT-15-OB UT-16-OB	This class represents the core user entity. Stores user credentials, identity, and even playlist relationships. Ensures user data is stored and retrieved correctly.
Java Domain Classes: Playlist Class	Playlist.java	UT-14-OB UT-17-OB	Manages the playlist of songs for a user. Data consistency needs to stay maintained. Ensures correct data modeling
Java Domain Classes: Song Class	Song.java	UT-18-CB UT-19-CB UT-20-CB	Handles all file parsing. Interacts with the JSON data.

Java Domain Classes: Song Filter Class	SongFilter.java	UT-21-OB UT-22-OB UT-23-OB UT-24-OB UT-25-OB	Correctly scores and validates songs based on the criteria provided to the SongFilter class and accurately scores them appropriately

IV. Integrated Tests

Test ID	Components Involved	Preconditions	Steps	Expected Result	Assigned Team Member
IT-01-TB	Auth Service Online		Start auth service with bun start	Server listening on port 3001	Devin
IT-02-TB	Spring Boot Backend Online		Compile and run Spring Boot app	Server listening on port 8080	Devin
IT-03-TB	Login Route	Testing user created	POST request to /auth/login	Returns signed JWT as a cookie	Devin
IT-04-TB	`/auth/me`	Testing user created and logged in	GET request to /auth/me	Validate JWT	Devin
IT-05-CB	User Write To File		Register a new user	User details should be added to JSON file	Devin
IT-06-TB	Password Hashing		Register a new user	User password should be hashed inside the JSON file	Devin
IT-07-CB	Add Song		Add a new song entry on the frontend	Frontend should send a POST request to backend, which should write to songs.json	Devin

V. Systems Tests

Test ID	Scenario	Preconditions	Steps	Expected Outcome	Assigned Team Member
ST-01-OB	New user registration	Server running, username unused	Enter username & password Submit registration	Account created successfully	N/A
ST-02-OB	Existing user login	Valid account existing	Enter valid credentials Login	Actress granted and session begins	N/A
ST-03-OB	Invalid login attempt	Account exists	Enter wrong password Login	Login denied with a clear error message	N/A
ST-04-OB	Authentication profile retrieval	User already logged in	Access “current user” function	Correct username and role information displayed	N/A
ST-05-OB	Unauthorized profile access	No valid token present	Attempt to access authentication	Blocked with no authenticated response/	N/A
ST-06-OB	Register Login `/auth/me`	Clean testing environment	Perform full sequence	End-end authentication flow succeeds	N/A
ST-07-OB	Admin role handling	One admin account and one regular user account exists	Login as both user and admin Retrieve profile	Role flag matches stored account	N/A
ST-08-OB	Music playlist management workflow	Java app supports the user and playlist objects	Create playlist Assign songs Retrieve playlist	The playlist reflects added songs correctly	Johnathan
ST-09-OB	Song metadata workflow	Availability of song	Load song from file Inspect fields Save once again	Song data carries correctly across the save cycle	Johnathan

System Testing Elaboration: The most critical system workflow is authentication because it secures

login and identity retrieval are essential project goals. The uploaded project description explicitly lists the registration, admin login, user login, playlists, and search/filtering as important features, so the testing plan prioritizes the workflows that already exist in code which then prepares for the rest in the final stages.

VI. Demonstration of Coverage

6.1 Coverage Mapped to User Stories

Authentication Domain

User Story	Feature Detail	Testing Coverage Included	Feature Status
S1 Registration	Create an account	UT-06-TB UT-07-TB IT-01 IT-05 ST-01-OB ST-06-OB	DONE
S2 Login	Securing login	UT-08-TB UT-09-TB UT-10-TB IT-02 IT-03 ST-02-OB ST-03-OB	DONE
S3 Logout	Logging out	UT-08-TB UT-09-TB UT-10-TB	DONE
S4 Admin Login	Admin authentication	ST-07-OB	DONE

Authentication Domain: *these tests ensure that the authentication workflows are validated, secure and have error-preventation, and this aligns with Streamly's goal of having a structured authentication backbone.*

Search & Filtering Domain

User Story	Feature Detail	Testing Coverage Included	Feature Status
S5 Seach by Title	Find songs by title	UT-21-OB UT-22-OB UT-24-OB	DONE

		UT-25-OB	
S6 Seach by artist	Find songs by artist	UT-21-OB UT-22-OB	DONE
S7 Seach by album	Find songs by album	UT-21-OB UT-22-OB	DONE
S8 Filter by Genre	Filter by genre	UT-21-OB UT-22-OB	IN-PROGRESS
S9 Filter by Year	Filter songs by release date	UT-21-OB UT-22-OB	DONE
S10 Filter by Trend	Filter songs by popularity	Planned for future tests	IN-PROGRESS

Search & Filtering Domain: *these features are classified as high priority in the previous planning documents (see Phase 1 and Phase 2 documents for more details) and are to receive more unit and integration testing in Iteration 3 once they have been fully implemented.*

Playlist & Favourites Domain

User Story	Feature Detail	Testing Coverage Included	Feature Status
S11 Favouritizing	Adding songs to favourites	UT-14-CB UT-17-OB IT-08 ST-08-OB	IN-PROGRESS
S12 Favorites Searching	Searching within favourites	ST-08-OB	LATER ITERATION
S13 Viewing Favourites	Viewing Favourites	Planned for future tests	LATER ITERATION

Playlist & Favourites Domain: *the current tests validate playlist data structures, which ensure overall correctness before the full GUI implementation and feature implementation.*

Admin Functionality

User Story	Feature Detail	Testing Coverage Included	Feature Status
S14 Adding Songs	Adding songs to database	UT-20 IT-09 ST-09	DONE
S15 Editing Songs	Modifying song metadata	Planned for future tests	DONE
S16 Removing Songs	Deleting songs form	Planned for future tests	DONE

	database		

Admin Functionality: *these tests are covered for Song.java (responsible for file parsing and saving) which directly support admin operations by making sure that the data can be safely written and retrieved.*

Nice-to-Have Features

User Story	Feature Detail	Testing Coverage Included	Feature Status
S17 Play Music	Music playback	Not in the scope for the current testing phase	DONE
S18 Password Encyrption	Secure password storage	UT-08 IT-02	DONE
S19 Searching Playlist	Searching inside a playlist	Planned for future tests	IN-PROGRESS

Nice to Have Features: *the password encryption is already partially covered through the login and registered tests using bcrypt for hashing, ensuring that all security requirements are met.*

6.2 High-Risk Components Coverage

These are the components that the Streamly team classified as high-risk and had to thoroughly test:

1. Authentication System (User Stories: S1-S4 & S18)
    - Includes password hashing, token-based sessions and JWT handling
    - Covered by multiple unit, integration and system tests (as seen in previous sections)

2. User-Playlist Relationships (S11-S13)
    - Ensures consistency of favourites and playlists
    - Covers all object-level and system testing

3. Data Persistence (JSON & Song Files)
    - json.ts: responsible for and handles user information storage.
    - Song.java: handles file parsing and saving

6.3 Distribution of Test Cases

Test Level	Number of Tests	Coverage Focus
Unit Tests	~25	Core methods, validation and

		logic
Integration Tests	7	Component Interaction
System Tests	9	End to end workflows

This distribution would ensure that there is strong method-level correctness, reliable user experience workflows, and verified component interaction.

6.4 Main Coverages of Testing Views

Testing View	Application
Clearbox CB	Song parsing, internal logic, JSON utilities
Translucent Box TB	Authentication routes and API behaviour
Opaque Box OB	User workflows (login, register, playlists)

All three testing views are intentionally distributed across unit, integration and system tests so that there is comprehensive understanding and validation of the system.

6.4 Gaps & Future Coverage

Based on this project plan, the following features have been somewhat implemented and are planned for future testing:

- Search functionality
- Filtering
- Playlist search
- Admin editing/removal

As these will be addressed in the next iteration — Iteration 3 Engagement Features.

By directly mapping all test cases to the initialized user stories and prioritizing the high-risk components, this testing plan ensures a comprehensive coverage of the current functionality and future system features, providing a clear confidence in the system's correctness, reliability and scalability.